

Wiskundige Samenvatting Van De Huidige Engine

Dit document beschrijft de exacte berekeningslogica van de huidige FuelPlan MVP zoals die nu in code staat. Het model is heuristisch: het is intern logisch consistent voor de demo, maar geen gevalideerd sportwetenschappelijk eindmodel.

1. Inputparameters

Parameter	Symbool	Verplicht	Opmerking
Gewicht	W	Ja	In kilogram
Geslacht	G	Ja	male of female
Leeftijd	A	Ja	In jaren
Hartslag	HR	Ja	Actuele hartslag in bpm
Vetpercentage	BF	Optioneel	Als leeg: fallback op basis van leeftijd + geslacht

Sanitizing / grenzen

```
W = clamp(round1(W), 35, 220)
A = clamp(round(A), 16, 90)
HR = clamp(round(HR), 40, 220)
BF = null of clamp(round1(BF), 3, 60)
```

2. Fallback-logica

Als vetpercentage ontbreekt, gebruikt de app een eenvoudige fallback.

```
BF_used = BF, als BF ingevuld is
BF_used = clamp(round1(12 + 0.18 * A), 10, 38), als G = male en BF leeg is
BF_used = clamp(round1(20 + 0.20 * A), 10, 38), als G = female en BF leeg is
```

De code bewaart ook een vlag: bodyFatSource = manual of fallback.

3. Intensiteit en max HR

```
HRmax_est = clamp(round(208 - 0.70 * A), 120, 210), als G = male  
HRmax_est = clamp(round(206 - 0.88 * A), 120, 210), als G = female
```

```
effortPct = clamp(round1((HR / HRmax_est) * 100), 30, 100)  
intensityScore = effortPct / 100
```

4. Lichaamssamenstelling

```
fatMassKg = round1(W * BF_used / 100)  
leanMassKg = round1(W - fatMassKg)  
leanMassFactor = clamp(leanMassKg / W, 0.45, 0.95)
```

5. Berekening Carbs Per Uur

```
carbPerHour = clamp(  
  round(  
    W * (0.34 + intensityScore * 0.42) * (0.82 + leanMassFactor * 0.28)  
  ),  
  20,  
  90  
)
```

Interpretatie

- Meer gewicht verhoogt de carbbehoefte.
- Hogere hartslag ten opzichte van geschatte max HR verhoogt de carbbehoefte.
- Relatief meer lean mass verhoogt de carbbehoefte.
- De uitkomst wordt hard begrensd tussen 20 en 90 g/uur.

6. Berekening Hydratie Per Uur

```
hydrationPerHour = clamp(  
  round(W * 5 + intensityScore * 320 + BF_used * 4),  
  350,  
  1000  
)
```

Interpretatie

- Meer gewicht verhoogt de vochtbehoefte.
- Hogere intensiteit verhoogt de vochtbehoefte.
- Hoger vetpercentage verhoogt in dit model licht de hydratatieschatting.
- De uitkomst wordt hard begrensd tussen 350 en 1000 ml/uur.

7. Dosis Per Reminder

De MVP gebruikt vaste reminderintervallen.

Type	Eerste reminder	Interval	Dosisformule
Carbs	25 min	20 min	$\text{carbDoseG} = \max(1, \text{round}(\text{carbPerHour} * 20 / 60))$
Hydratie	20 min	25 min	$\text{hydrationDoseMl} = \text{roundToNearestTen}(\text{hydrationPerHour} * 25 / 60)$

8. Tijdlijnlogica

8.1 Basisschema

```
carb base schedule = 25, 45, 65, 85, ...  
hydration base schedule = 20, 45, 70, 95, ...
```

8.2 Adaptieve planning

De uiteindelijke timeline is niet statisch. De volgende reminder verschuift op basis van echte intake-events en skips.

```
Voor elk event van een type:  
coveredIntervals = max(1, round(event.amount / doseAmount))  
  
start met nextReminderMinute = firstMinute  
voor elk event in tijdsvolgorde:  
    anchorMinute = max(nextReminderMinute, event.minute)  
    nextReminderMinute = anchorMinute + coveredIntervals * intervalMinute
```

Gevolg

- Te late intake schuift toekomstige waypoints naar later.
- Extra intake schuift toekomstige waypoints nog verder naar achter.

- Een skip telt als 1 verschoven waypoint, maar zonder echte intake in de balans.

8.3 Warning en due

```
phase = due, als elapsedMinute >= reminderMinute
phase = warning, als reminderMinute - elapsedMinute <= 3
phase = idle, anders
```

9. Fuel Balance

De fuel balance is continu per minuut, niet enkel op markerpunten.

```
carbTargetByNow = round(carbPerHour * elapsedMinute / 60)
hydrationTargetByNow = round(hydrationPerHour * elapsedMinute / 60)
```

```
actualCarbs = som van alle carb intake-events
actualHydration = som van alle hydration intake-events
```

```
carbBalance = actualCarbs - carbTargetByNow
hydrationBalance = actualHydration - hydrationTargetByNow
```

Interpretatie

- Na een intake stijgt de balans direct.
- Daarna daalt de balans geleidelijk opnieuw door het doorlopende verbruik.
- Een positieve balans betekent dat de atleet voorligt op plan.
- Een negatieve balans betekent dat de atleet achterloopt op plan.

10. Geschatte Afstand

De afstand is een displaywaarde en voedt de fueling-engine niet rechtstreeks.

```
leanFactor = clamp(1 - BF_used / 100, 0.55, 0.95)
paceKph = clamp(7.6 + effortPct * 0.075 + leanFactor * 2.1, 6.5, 18)
distanceKm = round1((paceKph * elapsedMinute) / 60)
```

11. Sessiedemo-constanten

```
simulationStep = 1 sec = 1 min
simulationTotal = 150 min
```

```
sessionStart = 08:00  
preAlertWindow = 3 min
```

12. Compacte Procesketen

Input parameters

- > sanitize and fallback
- > estimate max HR and intensity
- > derive body composition
- > calculate carbs/hour and hydration/hour
- > convert to per-reminder doses
- > build adaptive future reminder timeline
- > calculate continuous fuel balance
- > render watch state + guidance timeline

Belangrijke nuance: dit document beschrijft de huidige implementatie van de MVP. Het is een pragmatisch en intern consistent rekenmodel voor simulatie, geen medisch of wetenschappelijk gevalideerd protocol.